DevDocs — Settings / Configuration

1. Overview

DevDocs provides a global Settings / Configuration section where the user can configure the models and retrieval behavior used by the application.

The configuration is designed to be flexible without requiring DevDocs to maintain a predefined list of providers or model names.

The user can manually provide provider and model details where applicable.

Settings are **global for the single-user V1 application**.

---

2. Settings Structure

The Settings page contains four main sections:

```
Settings

├── LLM
├── Embeddings
├── Reranker
└── Advanced RAG
```

---

3. LLM Configuration

The user can manually configure the LLM used for answer generation.

Configuration

```
Provider
[ ____________________ ]

Model
[ ____________________ ]

API Key
[ ____________________ ]

Temperature
[ ____________________ ]
```

```
Max Output Tokens
[ ____________________ ]
```

Provider

The provider is entered manually.

DevDocs does not maintain a predefined provider dropdown for the LLM.

Examples could include providers such as Gemini or OpenRouter, but the UI should not be restricted to a fixed provider list.

Model

The user enters the exact model identifier supported by the selected provider.

No predefined model dropdown is required.

API Key

The user provides the API key required by the selected provider.

The API key should be displayed as a masked value by default.

A visibility toggle may be provided.

Temperature

Controls the randomness of LLM generation.

The user can modify the value within the range supported by the configured model/provider.

Max Output Tokens

Defines the maximum output-token safety ceiling for answer generation.

The LLM is still expected to generate responses adaptively according to the question.

This value acts as a technical maximum rather than a target response length.

---

4. Embedding Configuration

The user can manually configure the embedding provider and model used for project indexing and retrieval.

Configuration

```
Provider
[ ____________________ ]

Model
[ ____________________ ]

API Key
[ ____________________ ]
```

Provider

The user manually specifies the embedding provider.

Model

The user manually specifies the embedding model.

API Key

The user provides the API key required by the embedding provider.

---

5. Embedding Model Changes

Changing the embedding model requires the existing project embeddings to be regenerated.

However, changing only the embedding model does **not** require re-chunking the project.

Existing chunks are retained.

Process

Suppose the project currently uses:

```
Embedding Model A
        ↓
Project Files
        ↓
Chunks
        ↓
Embeddings A
```

The user changes to Model B:

```
Embedding Model B
        ↓
Existing Chunks
        ↓
Delete Embeddings A
        ↓
Generate Embeddings B
        ↓
Store Embeddings B
        ↓
Project Ready
```

Important

- The user does not need to upload the project again.
- Existing chunks are preserved.
- Existing embeddings are replaced.
- New embeddings are generated using the new model.
- The project must be re-indexed before the new embedding configuration is used for retrieval.

The re-indexing operation should use the already stored project files/chunks.

---

6. Reranker Configuration

DevDocs supports two reranker options:

```
Cross-Encoder
Jev
```

Default

Cross-Encoder is the default reranker.

DevDocs provides the default Cross-Encoder configuration internally.

The user does not need to provide the Cross-Encoder model or other technical configuration.

Configuration

```
Reranker Provider
[ Cross-Encoder ▼ ]

Top K
[ 5 ]
```

The default provider is Cross-Encoder.

---

7. Jev Reranker

The user can optionally select **Jev** instead of the default Cross-Encoder.

When Jev is selected, the user provides the required configuration.

Example:

```
Reranker Provider
[ Jev ▼ ]

Model
[ _____________________ ]

API Key
[ _____________________ ]

Top K
[ 5 ]
```

Behavior

When switching to Jev:

- Jev-specific configuration fields become available.
- The user provides the model/configuration details.
- The configured Jev reranker is used for subsequent retrieval operations.

When switching back to Cross-Encoder:

- Jev-specific fields are hidden.
- DevDocs returns to the predefined Cross-Encoder configuration.

---

8. Reranker Top K

`Top K` controls how many candidates remain after reranking.

Default:

```
5
```

The user may change this value through Advanced RAG configuration if desired.

The final number of chunks passed to the answer-generation context remains subject to the overall context/token budget.

---

9. Advanced RAG Configuration

Advanced RAG settings allow the user to tune retrieval behavior.

```
BM25 Top K
[ 10 ]

Vector Search Top K
[ 10 ]

Final Reranker Top K
[ 5 ]

Relevance Threshold
[ ___________ ]
```

---

10. BM25 Top K

Controls the number of candidates retrieved through BM25 keyword search.

Default:

```
10
```

BM25 is particularly useful for:

- Exact terms
- Function names
- Class names
- Variable names
- API endpoints
- Error messages
- Technical identifiers

---

11. Vector Search Top K

Controls the number of candidates retrieved through vector similarity search.

Default:

```
10
```

Vector search is primarily used for semantic retrieval.

---

12. RRF

Reciprocal Rank Fusion combines the BM25 and vector-search results.

RRF remains an internal retrieval mechanism.

The user does **not** configure the RRF algorithm itself.

The user only controls the retrieval limits that feed into it.

RRF parameters remain internal to DevDocs.

---

13. Final Reranker Top K

Controls the maximum number of candidates retained after reranking.

Default:

```
5
```

This represents the maximum number of final retrieved chunks available for answer generation.

The actual number may be lower depending on:

- Relevance
- Deduplication
- Context requirements
- Token budget

---

14. Relevance Threshold

Controls how relevant a retrieved result must be before it is considered suitable project context.

If retrieved results do not meet the configured relevance threshold:

```
No sufficiently relevant context
        ↓
```

```
Do not call answer-generation LLM
        ↓
Show project-context fallback
```

Example fallback:

> **I couldn't find relevant information in this project to answer that question.**

The exact threshold value can be tuned by the user.

---

15. Settings That Remain Internal

The following implementation details are **not exposed as user-facing settings in V1**:

- RRF parameters
- Chunk size
- Chunk overlap
- BM25 algorithm parameters
- Embedding dimensions
- Vector similarity metric
- Metadata filtering rules
- Duplicate-chunk removal rules
- Context token-budget calculations
- Conversation summarization threshold
- Automatic retry count
- Generation timeout
- Internal ingestion parameters
- Internal retrieval implementation details

These remain controlled by DevDocs.

The Settings page should not become an engineering/admin console.

---

16. Configuration Validation

DevDocs should validate user-provided configuration before it becomes active.

A **Test Connection** action should be available for configurations that require external provider access.

Example:

```
[Test Connection]    [Save]
```

Successful Validation

If the configuration is valid:

```
Connection successful
```

The user can then save the configuration.

Failed Validation

If validation fails, show a simple user-facing message:

> **Couldn't connect. Check your configuration and try again.**

Do not expose raw provider/API errors in the normal UI.

Technical errors can remain available through internal logs/debugging mechanisms.

---

17. Saving Configuration

When the user selects **Save**:

1. Validate the configuration where applicable.
2. Store the configuration.
3. Make the configuration active.
4. Apply it according to the configuration type.

Unsaved changes should not become active.

If the user leaves the Settings page without saving, unsaved changes are discarded.

---

18. Global Configuration

All settings are **global** in V1.

There are no separate per-project configurations.

```
Global Settings
      │
      ├── LLM
      ├── Embeddings
      ├── Reranker
      └── Advanced RAG
```

Any project uses the currently active global configuration.

---

19. Configuration Change Behavior

Different configuration types have different effects.

LLM Changes

Changing:

- Provider
- Model
- API key
- Temperature
- Max Output Tokens

takes effect for the **next query**.

No project re-indexing is required.

Reranker Changes

Changing:

- Cross-Encoder → Jev
- Jev model/configuration
- Reranker settings
- Reranker Top K

takes effect for the **next retrieval operation**.

No project re-indexing is required.

Advanced RAG Changes

Changing:

- BM25 Top K
- Vector Search Top K
- Final Reranker Top K
- Relevance Threshold

takes effect for the **next retrieval operation**.

Existing project data does not need to be reprocessed.

Embedding Changes

Changing the embedding provider or embedding model affects existing project embeddings.

The affected project's embeddings must be regenerated.

If only the embedding model changes:

```
Existing Chunks
      ↓
Keep
      ↓
Delete Old Embeddings
      ↓
Generate New Embeddings
      ↓
Store New Embeddings
      ↓
Project Ready
```

No re-upload is required.

No re-chunking is required.

---

20. Re-indexing After Embedding Changes

When an embedding configuration change requires re-indexing:

1. Detect that the embedding configuration has changed.
2. Notify the user that re-indexing is required.
3. Require explicit confirmation/action to start re-indexing.
4. Use the already stored project data.
5. Keep existing chunks.
6. Remove the project's old embeddings.
7. Generate embeddings using the new configuration.
8. Store the new embeddings.
9. Mark the project as ready when indexing completes.

The user should not have to upload the project again.

---

21. Cost / Usage Awareness

Because providers may have different pricing and usage limits, DevDocs should not silently trigger unnecessary model operations.

In particular:

- Embedding model changes should require explicit re-indexing.
- The user should be informed that re-indexing will regenerate embeddings.
- DevDocs should not automatically re-index every project immediately after a configuration field is edited.

This gives the user control over potentially expensive provider operations.

---

22. Security Considerations

API keys are sensitive configuration values.

DevDocs should:

- Mask API keys in the UI by default.
- Provide an optional visibility toggle.
- Avoid displaying API keys in logs.
- Avoid including API keys in error messages.
- Avoid exposing API keys to the chat interface.
- Store them securely according to the application's storage architecture.

---

23. Final Settings Structure

The complete V1 Settings page is:

```
Settings

LLM
├── Provider
├── Model
├── API Key
├── Temperature
└── Max Output Tokens

Embeddings
├── Provider
├── Model
└── API Key

Reranker
├── Provider
│   ├── Cross-Encoder (default)
│   └── Jev
├── Jev Model (when Jev selected)
├── Jev API Key (when required)
```

```
└── Top K

Advanced RAG
├── BM25 Top K
├── Vector Search Top K
├── Final Reranker Top K
└── Relevance Threshold
```

---

24. V1 Scope

Included

- Global LLM configuration
- Manual LLM provider/model configuration
- LLM API key
- Temperature
- Maximum output tokens
- Global embedding configuration
- Manual embedding provider/model configuration
- Embedding API key
- Cross-Encoder as default reranker
- Jev as optional reranker
- Jev-specific configuration
- Reranker Top K
- BM25 Top K
- Vector Search Top K
- Final Reranker Top K
- Relevance Threshold
- Configuration validation
- Test Connection
- Save/discard behavior
- Global configuration
- Embedding re-indexing when the embedding configuration changes
- Secure API-key handling

Not Included

- Per-project configuration
- Predefined LLM provider/model catalogs
- Predefined embedding provider/model catalogs
- User-configurable RRF internals
- User-configurable chunking internals
- User-configurable vector similarity implementation
- User-configurable metadata filtering
- User-configurable retry/timeouts
- Advanced engineering/debug configuration

---

25. Core V1 Principles

User-controlled models

The user can choose the provider and model rather than being restricted to a hard-coded model list.

Sensible defaults

DevDocs provides sensible defaults for internal components such as the Cross-Encoder.

Global configuration

Settings apply globally across the single-user application.

No unnecessary reprocessing

Only configuration changes that affect indexed data trigger re-indexing.

Explicit expensive operations

Embedding regeneration requires an explicit user action rather than happening silently.

Minimal configuration surface

Only settings that provide meaningful user control are exposed.

Implementation details remain internal to DevDocs.